

C# 言語とは

C# は、Web アプリケーション、Windows アプリケーション、ゲーム開発、モバイルアプリ開発など、幅広い分野で活用されているプログラミング言語です。

C# はマイクロソフトが開発しているので、特に Windows 環境での開発に強みがあり Microsoft 製品との連携が非常にスムーズです。近年 C# を扱う環境ではクロスプラットフォーム対応が強化されています。

C# は**オブジェクト指向プログラミング**で、**Java 言語**と多くの共通点を持っています。しかし、完全に同じというわけではなくそれぞれに異なる特徴や強みがあります。

C# に慣れることで Java の習得も早まる傾向にあります。

C# 言語の特徴

C# にまつわるツールに因らず、言語そのものが持っている特徴

オブジェクト指向プログラミング

再利用性や保守性が高い

型の安全性

コードの信頼性 / エラー未然防止

具体的な内容については学習を進めながら徐々に理解していきます

プログラミング言語と統合開発環境 (IDE)

プログラミング言語は単体では何もできない

ただそこに書かれているというだけ

```
namespace d01_sample1
{
    internal class Program
    {
        static void Main(string[] args)
        {
            // 1. 基本的な変数の宣言と初期化
            int age = 25; // 整数型
            double height = 175.5; // 浮動小数点型
            bool isMarried = true; // 真偽値型
            char initial = 'T'; // 文字型
            string name = "山田太郎"; // 文字列型 (参照型)

            Console.WriteLine($"名前: {name}");
            Console.WriteLine($"年齢: {age}歳");
            Console.WriteLine($"身長: {height}cm");
            Console.WriteLine($"既婚者: {isMarried}");
            Console.WriteLine($"イニシャル: {initial}");

            Console.WriteLine("~~~~~ var キーワードの利用 ~~~~");
            var quantity = 100; // 明示して推測される (変につかないdoubleになるので注意!)
            var price = 99.99m; // decimalとして推測される (変につかないdoubleになるので注意!)
            var message = "Hello CAT!"; // stringとして推測される

            Console.WriteLine($"数量: {quantity}");
            Console.WriteLine($"価格: {price}円");
            Console.WriteLine($"メッセージ: {message}");

            // 推測された型を確認
            Console.WriteLine($"quantityの型: {quantity.GetType()}");
            Console.WriteLine($"priceの型: {price.GetType()}");
            Console.WriteLine($"messageの型: {message.GetType()}");
        }
    }
}
```

プログラミングで
書かれた命令を解釈

実行するツール
(ランタイム)

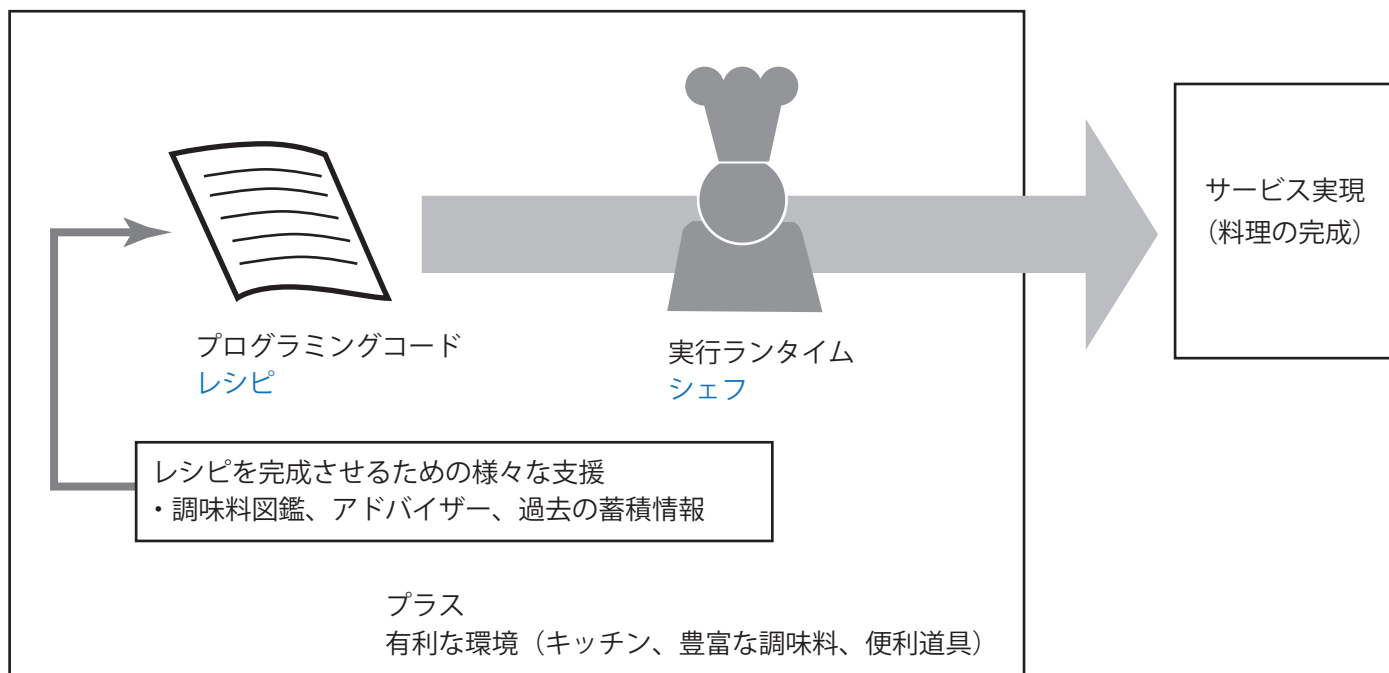
サービス実現

そもそもプログラミング言語を
書くのはなかなかタイヘン

色分けして見やすい
優秀な予測変換
便利なショートカットがあるツール
デバッグが充実

サポート

料理に例えると…？

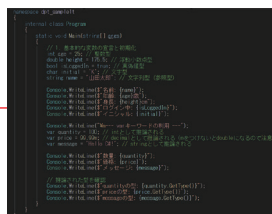


制作から再生（実行）まで一貫して作業できる環境
統合開発環境 (IDE: Integrated Development Environment)

C# を扱える**統合開発環境 (IDE)** ⇒ VisualStudio

VisualStudio

- **コードエディター**
- IntelliSense: 入力支援
- デバッグ機能
- GUI デザイナー: ドラッグ & ドロップで画面設計
- プロジェクト管理機能: 複数の制作物をまとめておける



実行

.NET SDK (Software Development Kit)

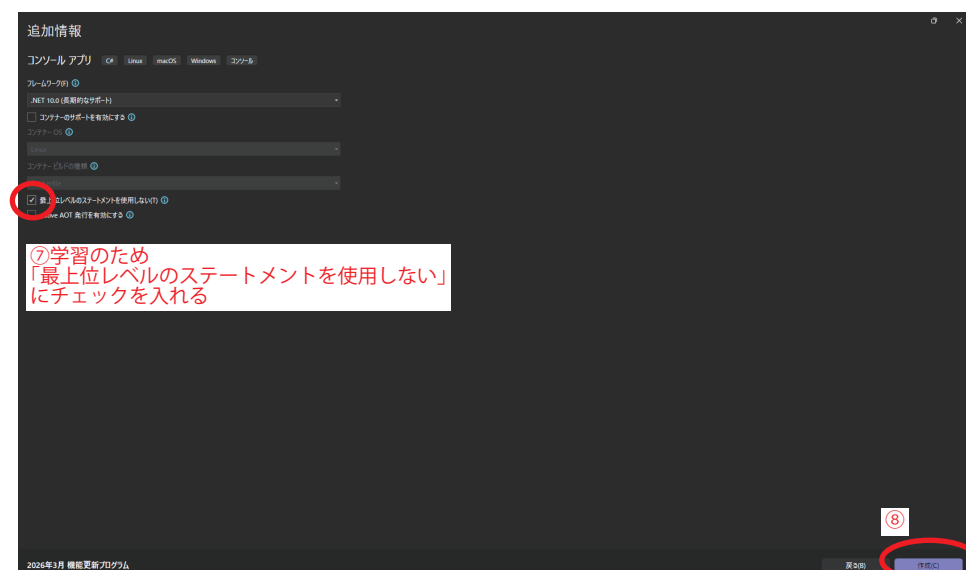
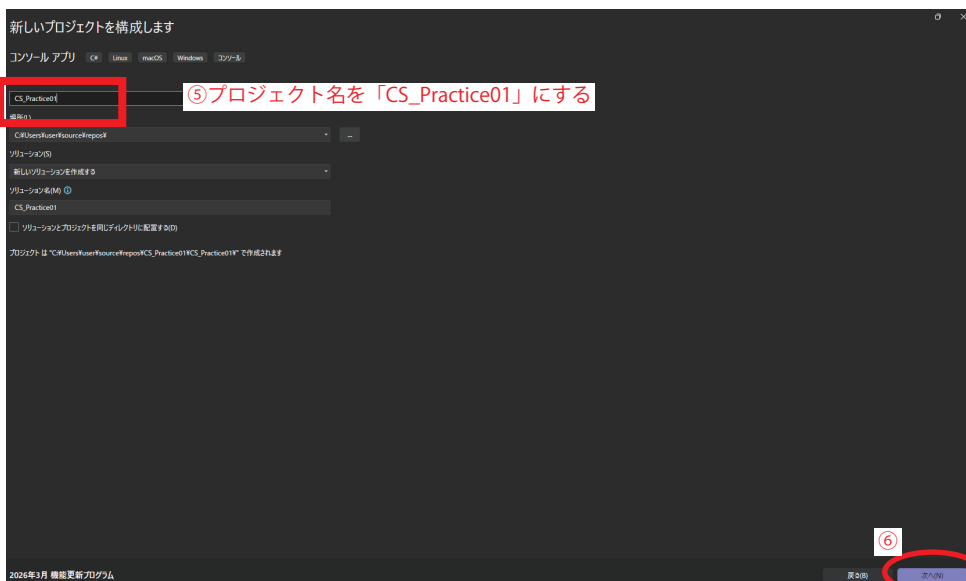
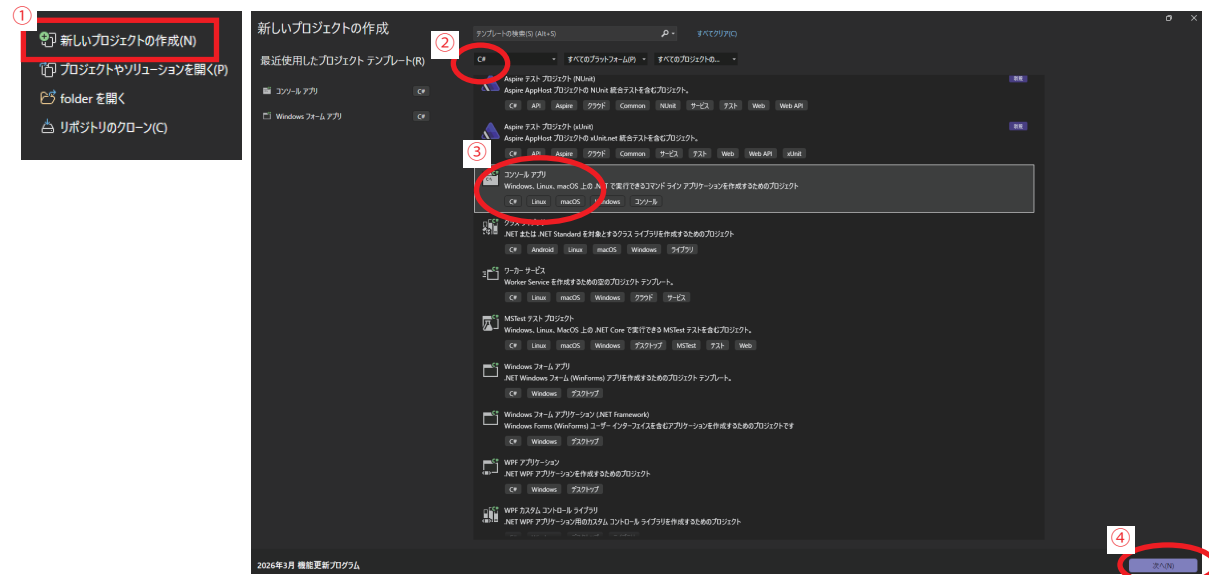
- C# コンパイラ: プログラムを**中間言語 (CIL: Common Intermediate Language)** にコンパイル
- .NET ライブラリ (BCL: Base Class Library)

.NET ランタイム (CLR: Common Language Runtime)

- JIT コンパイラ: **機械語に変換しながら実行**
- ガベージコレクション: メモリ管理
- クロスプラットフォーム対応機能

VisualStudio で実行してみましよう

VisualStudio の機能を使って、実際にソースコードを記述、再生するとどうなるのかを確かめてみましょう。

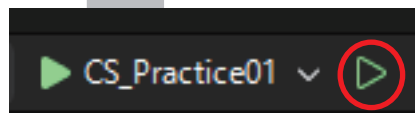


最上位レベルのステートメント

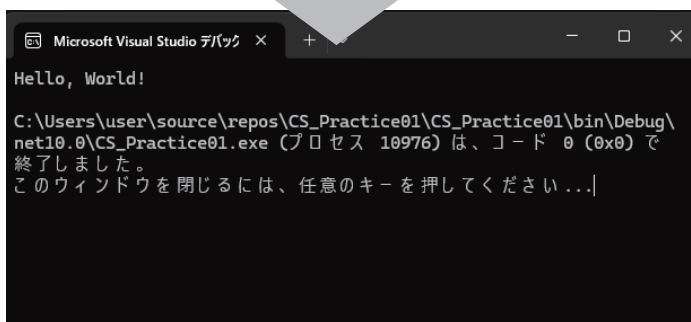
従来の必要だった記述を省略し、プログラムの重要な部分のみで構成できる機能

※授業の序盤では学習のためにあえて省略しない（ステートメントは使わない）

```
Program.cs
CS_Practice01
namespace CS_Practice01
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```



デバッグなしで再生



コンソールが立ち上がり、
結果が表示されていれば OK

※結果を見終わったら
何かキーをおして
コンソールは閉じましょう

VisualStudio のコードエディター上にソースコードを記述して再生ボタン
⇒VisualStudio 内部でいろいろな工程を経て機械語に変換実行され、結果が表示

ソースコード…プログラミング言語でエディタに記述された命令

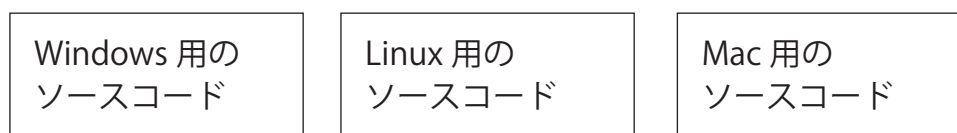
再生が終わったらファイルを保存します (ファイル→「Program.cs の保存」)
基本的には保存は常にショートカットを使いましょう (Ctl キー + S キー)

共通中間言語 (CIL : Common Intermediate Language)

.NET における実行完了までの流れ



なぜ中間言語に変換?? 逆に中間言語がないと?

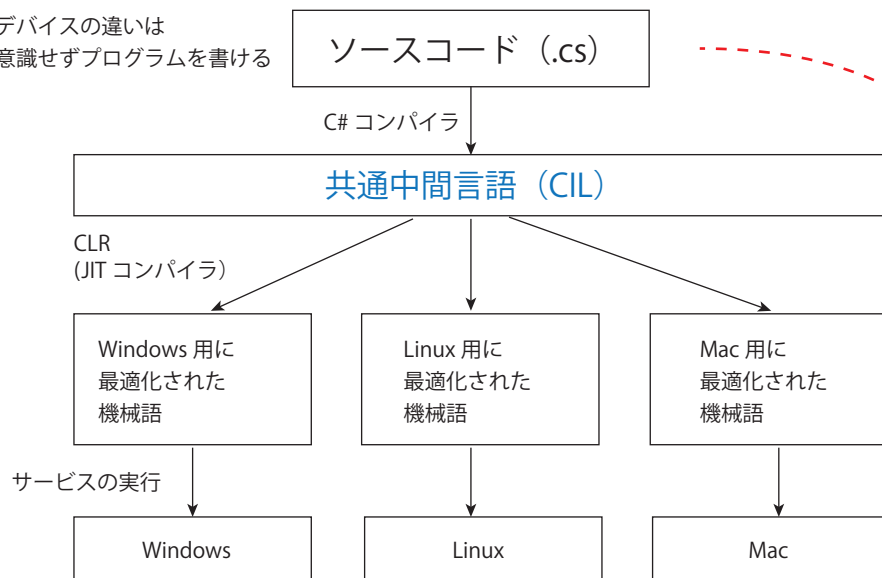


ソースコード上ではデバイスの違いを気にしない

中間言語というクッション ⇒ 機械語にコンパイルされる際、実行デバイス合わせて最適化

コンパイル…コードを一括で別の形式に変換する

デバイスの違いは
意識せずプログラムを書ける



直接ソースコードを最適化するより中間言語 (IL) を設けた方が柔軟で効率的な場合が多い

プログラミング言語そのものも重要ですが、プログラミング言語を解釈し実行するツールも大切な存在であるということを知り、関心を寄せていきましょう。

C# 言語を取り扱える主な統合開発環境 (IDE)



Visual Studio と Visual Studio Code は別物！



Visual Studio Code は Microsoft 社が提供する「テキストエディタ」豊富な拡張機能で IDE 的に匹敵する使い方ができる

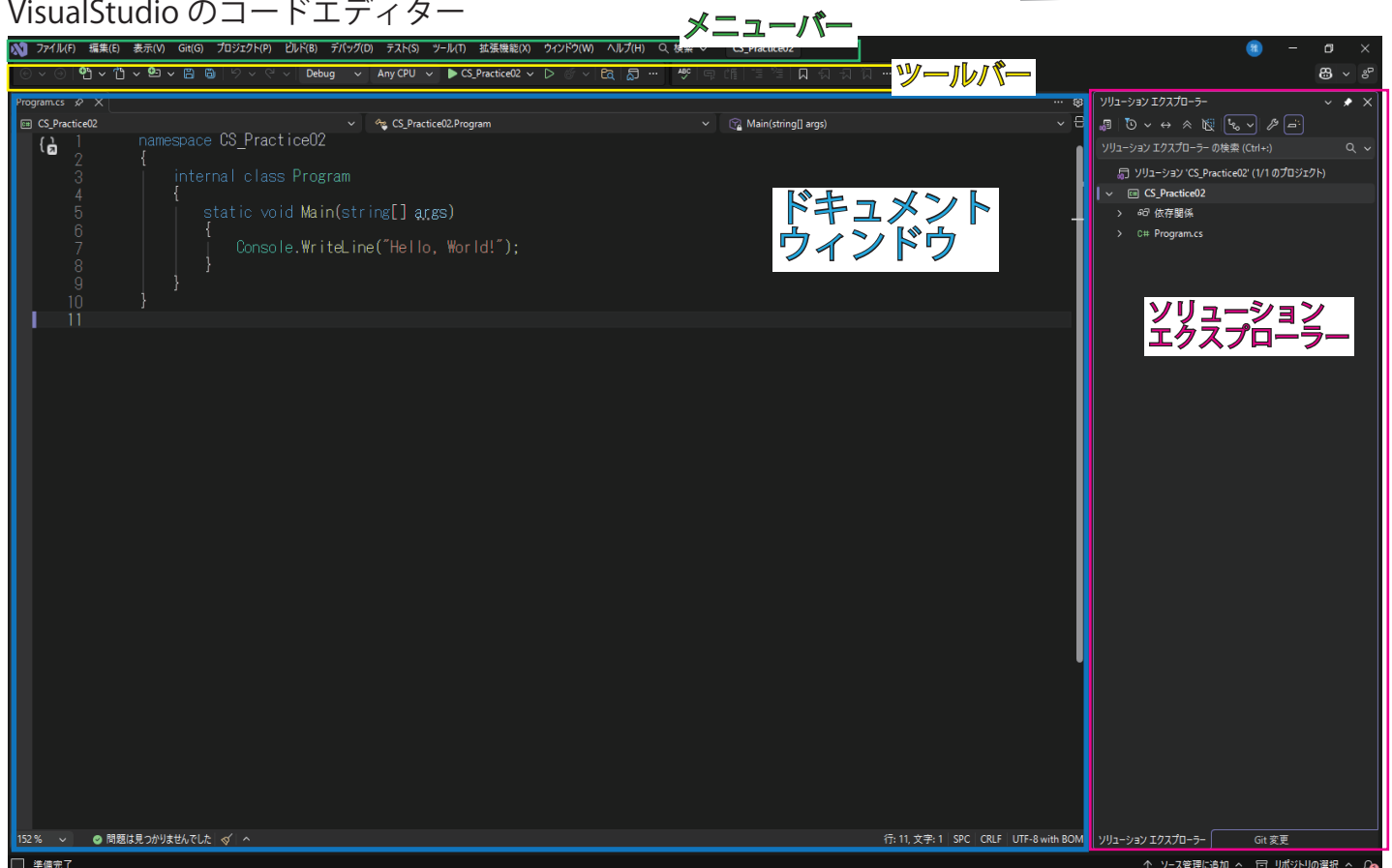
プロジェクトの作成とビルド・実行

プロジェクト・ソリューション・クラス

VisualStudio では複数のプロジェクト・ファイルを管理できます。
まずは2つ目のプロジェクトを作成しながら、あらためて構造を学んでいきましょう。

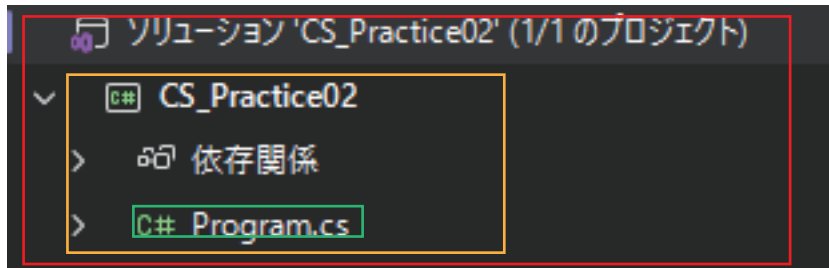


VisualStudio のコードエディター



ソリューションエクスプローラー

生成したプロジェクトやファイル、関連要素などを階層構造で管理するウィンドウ

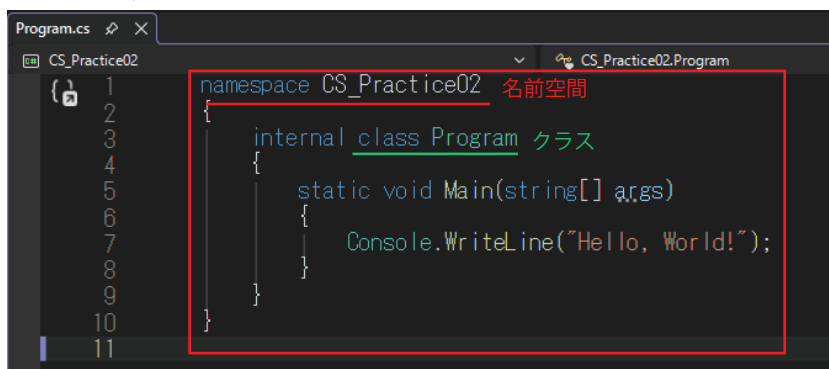


ソリューション…問題解決のために複数プロジェクトをグループ化して管理

プロジェクト…サービスやアプリの単位 ※新規作成時はこの名前を決める

クラス…アプリ構築に必要なデータとデータ操作機能をまとめたもの

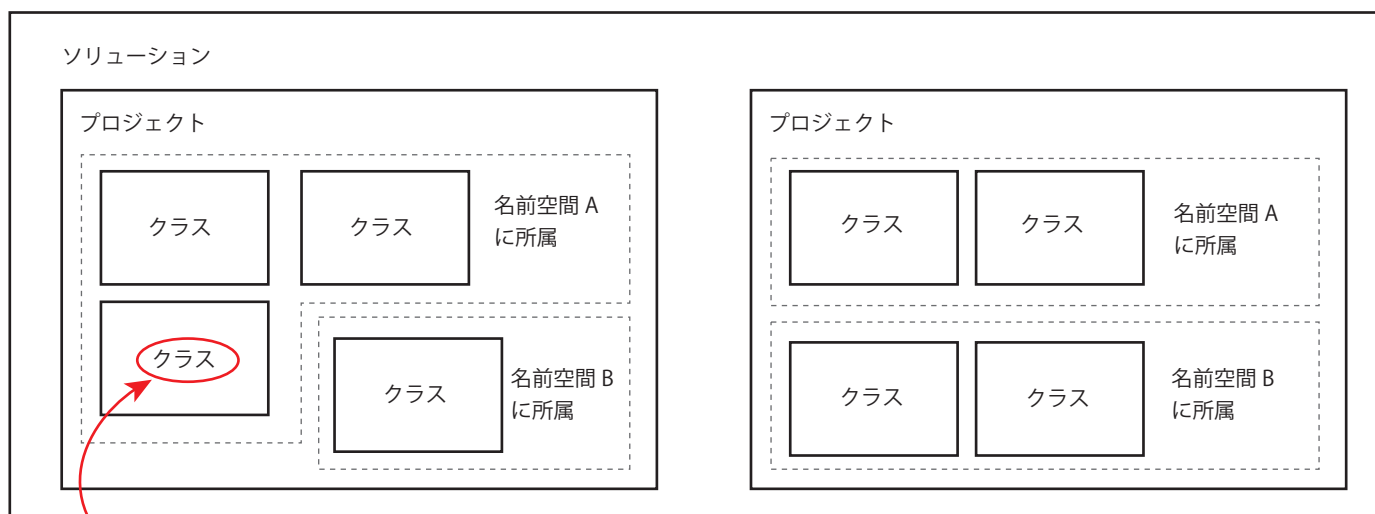
依存関係…プロジェクト動作に必要な外部ライブラリやフレームワークを示している



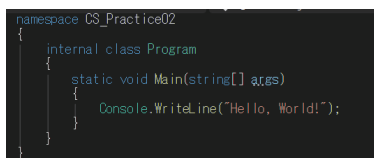
プロジェクト名を決めて新規生成すると
・プロジェクトと同名のソリューション (.sln)
・Program という名前のクラス (.cs)
が自動で生成されます。

生成されたばかりのクラスでは、
プロジェクトの内部でどのグループに所属しているかが自動で決まり、通常は
プロジェクト名と同名のグループ (名前空間と呼ぶ) に所属しています。

もう少し構造を深掘してみましょう

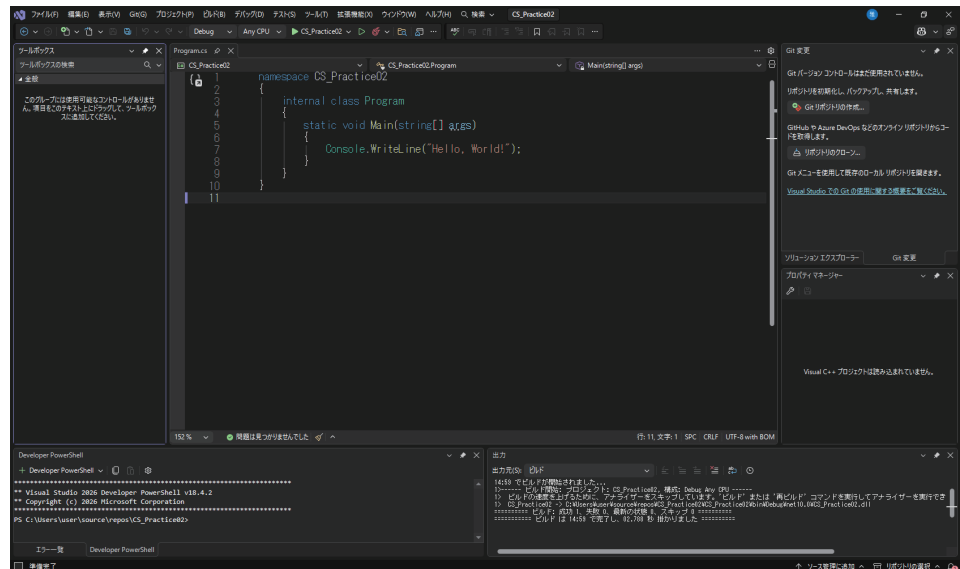
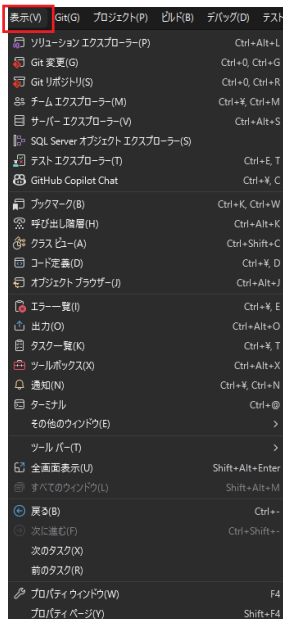


目的に必要なだけクラスを生成し、連携させていきます。
我々はクラスの内容をプログラミング言語で記述していくことになります。

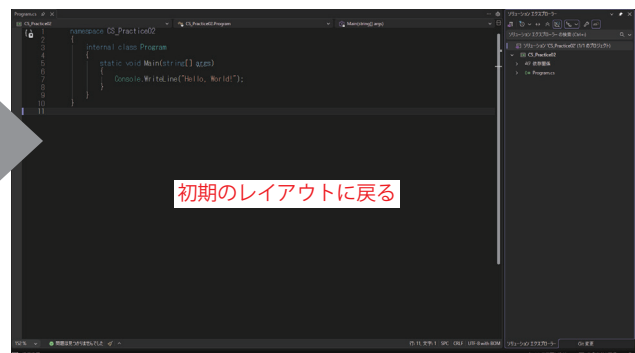
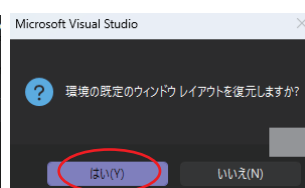
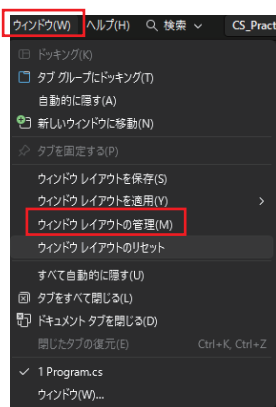


生成された Program クラスには、サンプルコードが載っています
クラスの内容：「コンソールに Hello World と表示する」

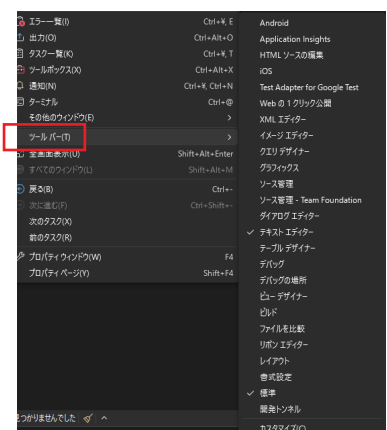
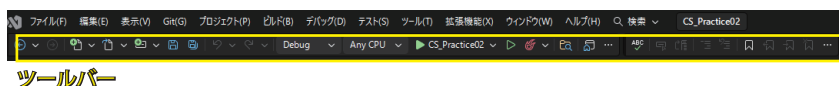
VisualStudio のコードエディター内の各ウィンドウは、ドラッグ操作で自由なレイアウトに組み替えてることができます。
不要なものは「×」で閉じたり、“表示メニュー” から不足しているウィンドウを追加してカスタマイズ可能です。



カスタマイズでごちゃごちゃになったり、間違っって触ってよくわからなくなってしまった時は “ウィンドウメニュー” から「ウィンドウレイアウトのリセット」を行いましょう。



ツールバーの配置は「ウィンドウレイアウトのリセット」ではコントロールできないので注意しましょう。ツールバーの表示物は “表示メニュー” の “ツールバー” から選定し、ドラッグ操作で好みに配置します。

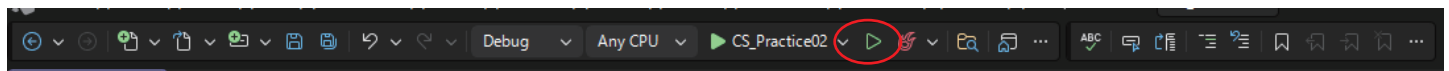


ソースコードの編集と実行

ソースコードを次のように再編集して実行しましょう。

※〇〇の部分に自分の名前を入れてみましょう。

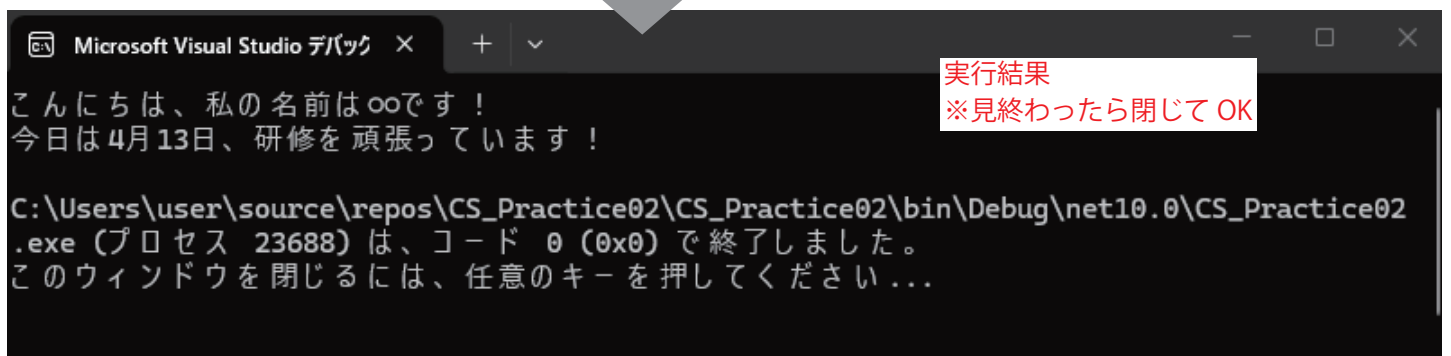
```
namespace CS_Practice02
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.Write("こんにちは、私の名前は");
            Console.WriteLine("〇〇です!");
            Console.Write("今日は4月13日、");
            Console.WriteLine("研修を頑張っています!");
        }
    }
}
```



ツールバーの「デバッグなしで再生」アイコンを押して実行

(ショートカットは Ctl + F5 キー)

※Function キーが効かない人は「Fn キー」+「Esc キー」でモード切替



Console.Write(文字列); ... () の中の文字列をコンソールに表示、**改行しない**

Console.WriteLine(文字列) ... () の中の文字列をコンソールに表示したあと、**改行する**

Console.WriteLine などのように、何かが実行される操作のことを
メソッド (文脈によっては関数) と呼びます。

ソースコード上で**文字列の表現**は (") **ダブルクォーテーション** で囲む

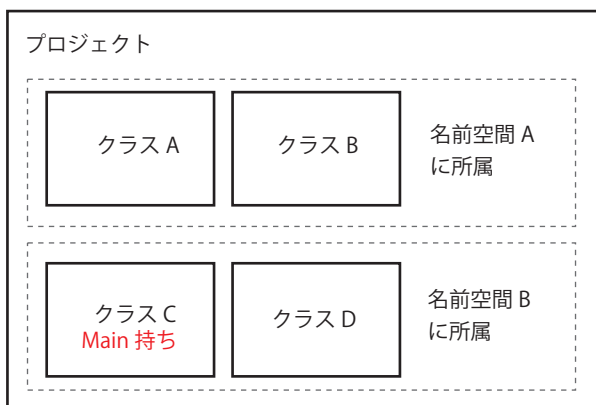
Main メソッド (関数) は消してはいけない！

自動生成されたソースコードには

`static void Main (String[] args)`

という謎の部分が膨れていますが、これは何でしょう？

これはプログラムが再生された時に一番最初に呼び出されるポイントで、この Main メソッドを起点に処理が連鎖していきます。この開始地点を「エントリポイント」と呼びます。プログラムはエントリポイントがないと稼働させることができません。エントリポイントは原則ひとつのプロジェクトにひとつだけ設置します。



例) 左の図の場合
プログラムを再生すると、
まずクラス C の Main メソッドに書かれてい
る内容が始まる

クラス C の Main メソッドから
必要に応じて他クラス的能力が呼び出される

すべての事柄は Main メソッドを中心に行われる (処理を {} で囲む必要がある)

```
1 namespace CS_Practice03
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             Console.WriteLine("Hello, World!");
8         }
9     }
10 }
```

名前空間

クラス

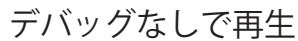
Main

Main メソッドの {} の中に処理を書かないといけないので注意！

初心者のエラー原因の 8 割はコレ！

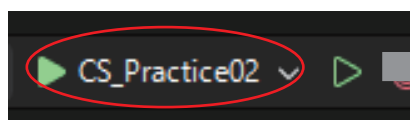
- {} の記号がそろっていない (間違ってどちらかを削除してしまっている)
- Main の {} の外に処理が書かれている
- 大文字、小文字が正しくない / スペルミス

見直し・点検を怠らない人はプログラムの理解度が
高まっていく傾向にあります。皆さんはどうでしょうか？



デバッグありで再生は「デバッガー」と呼ばれる観察・分析ツールを付与しながらプログラムを実行します。

左の枠を
クリックすることで
ブレークポイントを
好きにつけ外し



Program.cs

```

1 namespace CS_Practice02
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             Console.WriteLine("こんにちは、私の名前は");
8             Console.WriteLine("〇〇です!");
9             Console.WriteLine("今日は4月13日、");
10            Console.WriteLine("研修を頑張っています!");
11        }
12    }
13 }
14

```

args: {string[0]}

Watch (Ctrl+D)

名前	値	種類
args	(string[0])	表示 - string[]

Debug Console (Ctrl+O)

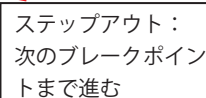
表示 (Ctrl+F)

名前

CS.Practice02.CS.Practice02.Program.Main(string[] args) 行 7

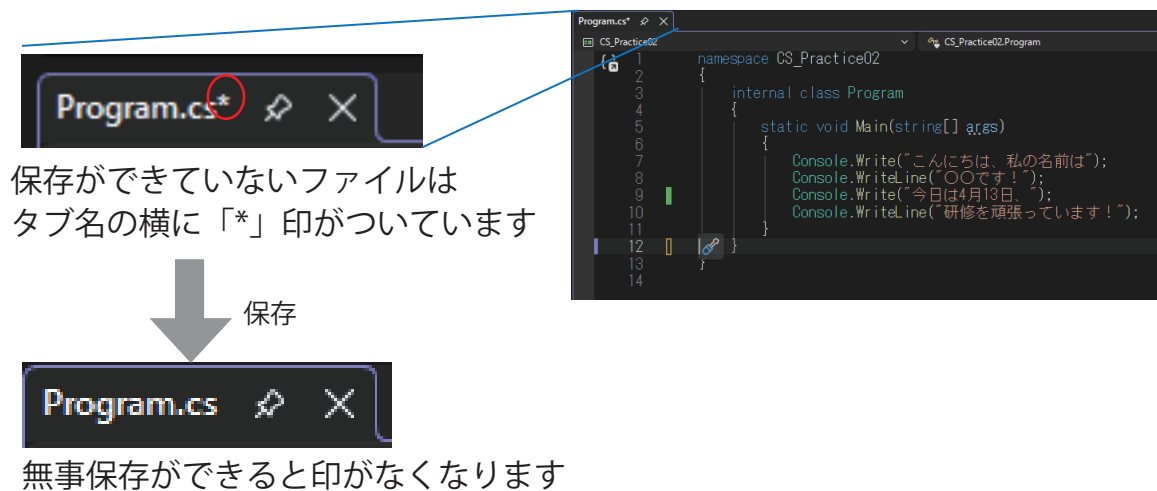
表示 - string[]

再生中、ツールバーには専用アイコンが並びます

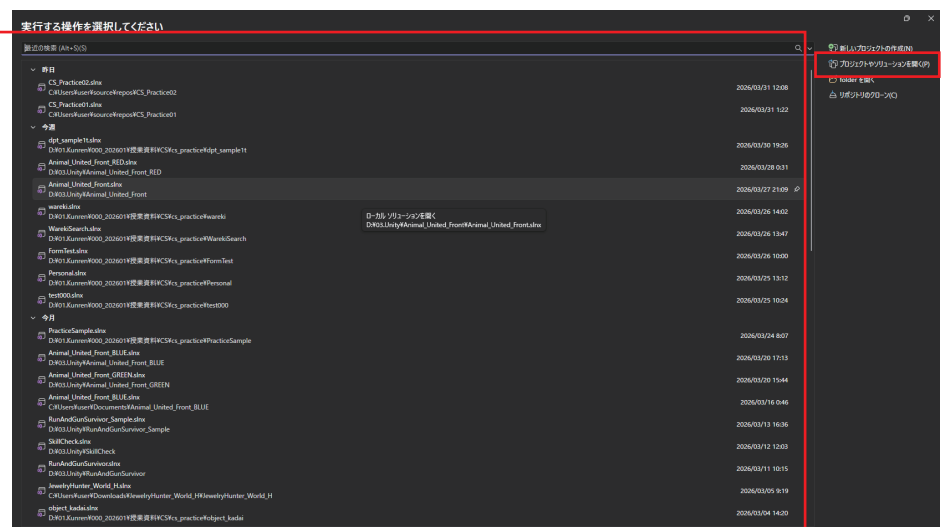
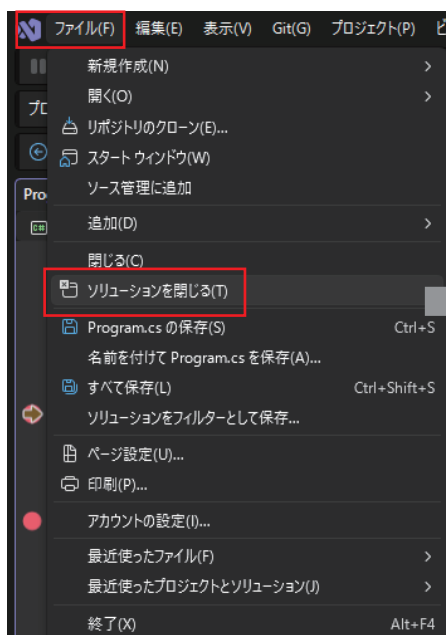


11

再生が終わったらファイルを保存します (ファイル→「Program.cs の保存」)
基本的には保存は常にショートカットを使いましょう (Ctrl キー + S キー)



用事がなくなったソリューションはファイルメニューから
「ソリューションを閉じる」 (または Visual Studio ごと閉じる)



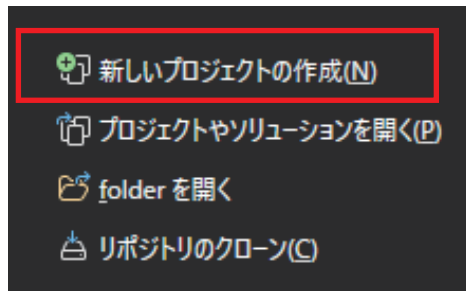
トップページに過去ソリューションが履歴掲載されます。
もう一度編集する場合は
「プロジェクトやソリューションを開く」の他、
この履歴をクリックしても OK です。

演習課題

それぞれの指示に従い「プロジェクト作成」→「ソースコードの模写」→「デバッグなしで実行（またはデバッグありで実行）」を行っていきましょう。

演習課題 1-1

- ① トップ画面の「新しいプロジェクトの作成」から「C# コンソールアプリ」のプロジェクトとして「CS_Practice03」を作成しましょう。



ソースコード「Program.cs」が生成されたら次の画像のように再編集して、保存しましょう。
※タイピングや、記号の打ち込み、スペル間違いをしないようにする練習も兼ねています。
コードの中身についてはまだ理解できなくても大丈夫です。

- ② 編集が終わったら「デバッグなしで再生」でエラーが出ないかをチェック、エラーがある場合は書き間違いがないか見直ししましょう。

- ③ エラーなく実行ができれば、好みの場所にブレークポイントを設けて「デバッグありで再生」も試してみましょう。

- ④ ファイルメニューの「ソリューションを閉じる」でトップ画面に戻りましょう。

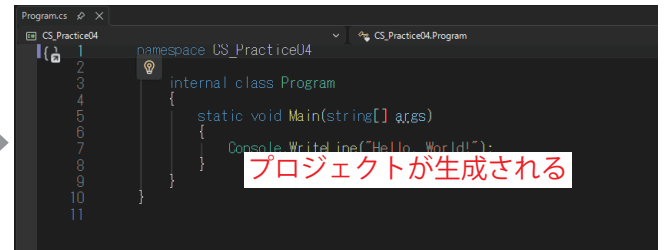
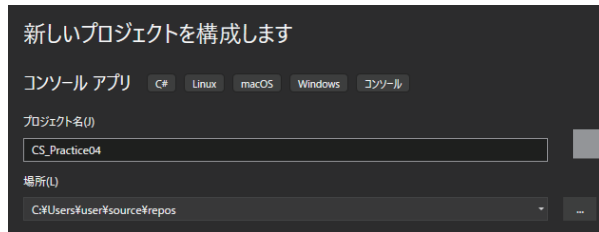
```
1 namespace CS_Practice03
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             //時間と分に関する変数を用意
8             int hours = 9;
9             int minutes = 30;
10
11             //研修講座の時間をコンソールに表示
12             Console.WriteLine("研修講座は" + hours + "時" + minutes + "分からです");
13
14             //変数hoursに時間を追加する
15             hours = hours + 9;
16
17             //分を0にする
18             minutes = 0;
19
20             //終業時刻をコンソールに表示
21             Console.WriteLine("業務は" + hours + "時" + minutes + "分までです");
22         }
23     }
24 }
```

演習課題 1-2

演習を通してプロジェクトを削除する方法を学習しましょう。

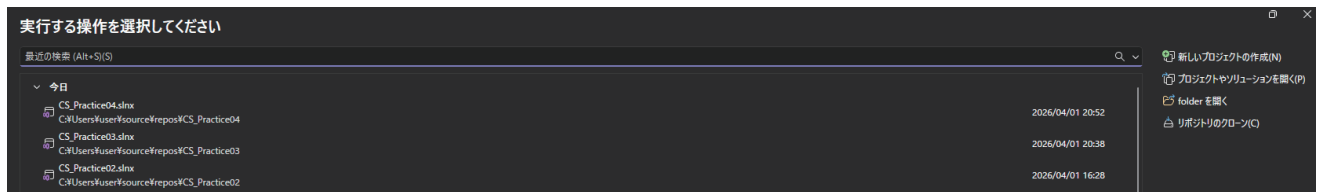
①まずはトップ画面から

「C# コンソールアプリ」のプロジェクトとして「CS_Practice04」を作成しましょう。



プロジェクトの保管場所を意識しておくこと。
※パソコンにより途中の記述が違う可能性があります

②「CS_Practice04」をファイルメニューの「ソリューションを閉じる」で閉じ、トップ画面に戻りましょう。

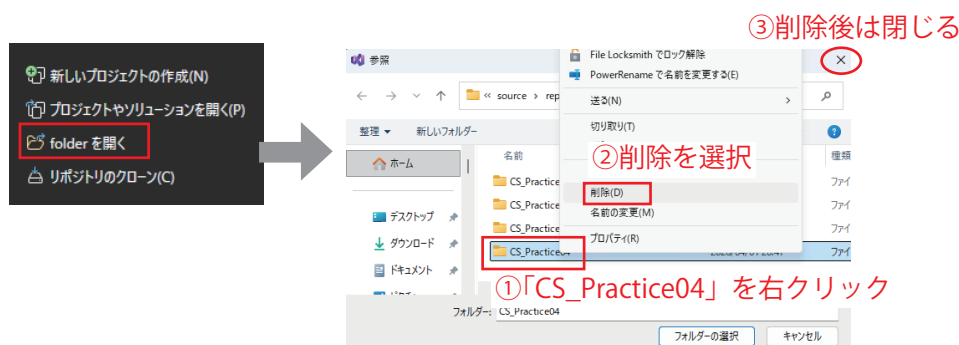


プロジェクトの削除には 「データそのものを削除」「トップ画面のリストから除外」が必要です

③-1 データそのものを削除

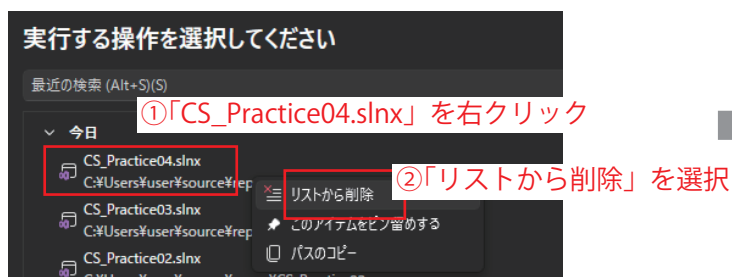
トップ画面サイドメニューの「folderを開く」をクリックしましょう。

Windows のエクスプローラーにてプロジェクトの保管場所が示されるのでデータを削除します。



③-2 トップ画面のリストから除外

トップ画面の履歴リスト「CS_Practice04」を右クリック、「リストから削除」を選びます。

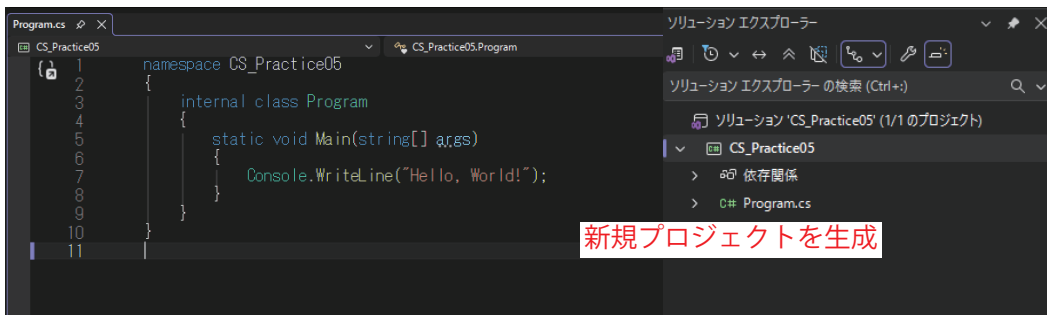


リストからもなくなれば OK
※以降も削除は慎重に！

演習課題 1-3

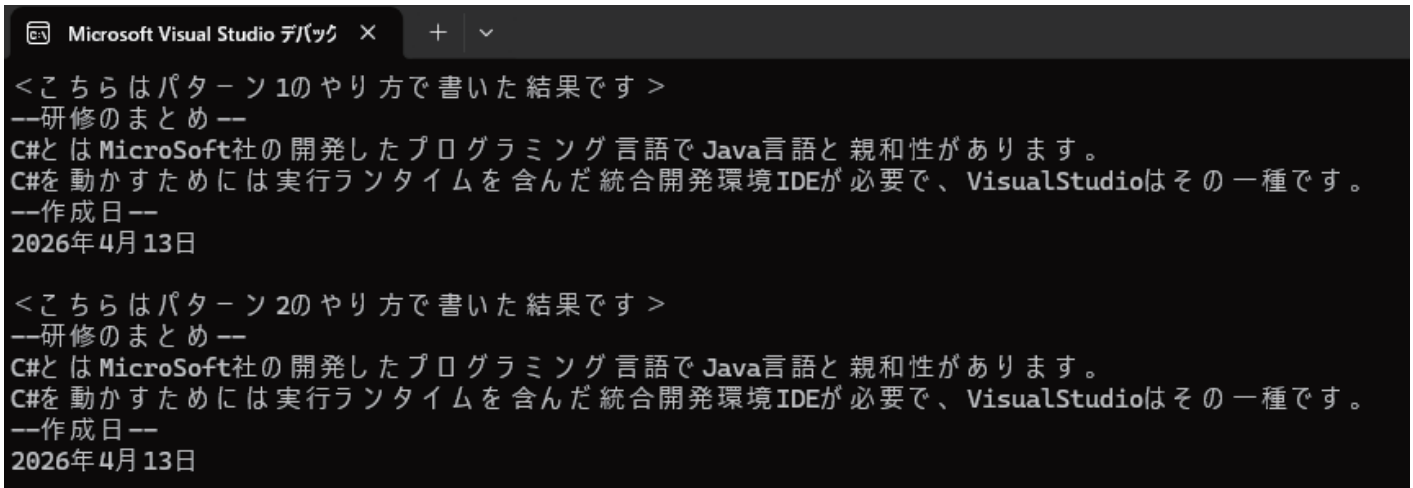
ここでは、となるソースコードを用意して再生結果が同じであることを知ります。
結果が同じでも様々な書き方があり、より適切な書き方が何かを考えてもらいます。

- ① トップ画面から「C# コンソールアプリ」として
新規プロジェクト「CS_Practice05」を作成しましょう。



- ② 生成された Program.cs の Main メソッドの中身を次の画像と同じになるように
再編集しましょう ({} の位置、大文字・小文字、スペルミスに注意)

```
1 namespace CS_Practice05
2 {
3     internal class Program
4     {
5         static void Main(string[] args)
6         {
7             //変数の用意
8             string title1 = "研修のまとめ";
9             string title2 = "作成日";
10
11             //パターン1のはじまり
12             Console.WriteLine("<こちらはパターン1のやり方で書いた結果です>");
13             //見出し1
14             Console.Write("--");
15             Console.Write(title1);
16             Console.Write("--");
17             //ダブルクォーテーションの囲みの中に何も書かないことによって改行する
18             Console.WriteLine("");
19             //文章の記述
20             Console.WriteLine("C#とはMicrosoft社の開発したプログラミング言語です。");
21             Console.WriteLine("C#を動かすためには実行ランタイムを含んだ統合開発環境IDEが必要です。");
22             //見出し2
23             Console.Write("--");
24             Console.Write(title2);
25             Console.Write("--");
26             Console.WriteLine(""); //改行
27             Console.WriteLine("2026年4月13日"); //文章の記述
28             Console.WriteLine(""); //改行
29             Console.WriteLine(""); //改行
30
31             //パターン2のはじまり
32             Console.WriteLine("<こちらはパターン2のやり方で書いた結果です>");
33             //見出し1
34             Console.WriteLine("--" + title1 + "--");
35             //文章の記述
36             Console.WriteLine("C#とはMicrosoft社の開発したプログラミング言語です。");
37             Console.WriteLine("C#を動かすためには実行ランタイムを含んだ統合開発環境IDEが必要です。");
38             //見出し2
39             Console.WriteLine("--" + title2 + "--");
40             //文章の記述
41             Console.WriteLine("2026年4月13日");
42         }
43     }
44 }
```

結果は同じですが、パターン 1 とパターン 2 のどちらがよりスマートに感じましたか？

コメント機能

ソースコードには「メモ」となるコメントをつけることができます。
コメントは実際のプログラムの再生には影響を及ぼしません。

C# 言語におけるコメントは 2 種類あります

- ・行ベースのコメント「//」

```
// 単体のコメントになります  
Console.WriteLine("文字列"); //このようにコードの後ろにコメントを始めることもできます
```

- ・複数行を囲めるコメント「/* */」

```
/*  
ここにコメントを書きます。  
複数行の内容をまとめて囲むことができます。  
*/  
  
Console.WriteLine("文字列"); /* 単独行や、短い内容でも可能です */
```

コメントとは、**他人や将来の自分に「コードの意図（Why）」を伝えるための**不可欠なメモです。

作成したその瞬間ではわかっているけど、少し時間をおくと自分自身でもどのような意味をもったソースコードなのか、わからなくなるのが自然です。ましてや**自分以外の者**もメンテナンスするものであるなら、なおさら**道しるべとなるメモが不可欠**です。

コメント（メモ）を適切につけておく気遣いは、エンジニアに必要なマナーです